49.1.1 Other ways of modeling

Instead of capturing every single person in code, a 'contact network' model, it is possible to use an ODE approach to disease modeling. You would then model the percentage of infected persons by a single scalar, and derive relations for that and other scalars [2, 20].

This is known as a 'compartmental model', where each of the three SIR states, Susceptible, Infected, Removed, is a compartment: a section of the population. Both the contact network and the compartmental model capture part of the truth. In fact, they can be combined. We can consider a country as a set of cities, where people travel between any pair of cities. We then use a compartmental model inside a city, and a contact network between cities.

In this project we will only use the network model.

49.2 Coding

The following sections are a step-by-step development of the code for this project. Later we will discuss running this code as a scientific experiment.

Remark *In various places you need a random number generator. It is preferred that you use the C++ native one (section 24.7) rather than one from the C language.*

49.2.1 Person basics

The most basic component of a disease simulation is to infect a person with a disease, and see the time development of that infection. Thus you need a person class and a disease class.

Before you start coding, ask yourself what behaviors these classes need to support.

- Person. The basic methods for a person are
  1. Get infected;
  2. Get vaccinated; and
  3. Progress by one day.

  Furthermore you may want to query what the state of the person is: are they healthy, sick, recovered?
- Disease. For now, a disease itself doesn't need any methods. (Later in the project you may want it to have a method `mutate`.) However, you may want to query certain properties:
  1. Chance of transmission; and
  2. Number of days a person stays sick when infected.

Let's start with some simple tests. First, if you have a disease with a transmission probability of $0 < p \leq 1$, then infecting N people should result in roughly pN infections.

Sample code:

```
// infect/contain.cpp
Person joe;
joe.contact(flu);
if (joe.is_infected())
  ninfected++;
```

Exercise 49.1. Write `Disease` and `Person` classes with enough methods to test infection.

Test with $p = 1$ and $p < 1$.

Next, once a person gets infected, the disease should run its course in the prescribed number of days.

Sample code where a person gets infected with a disease:

```
// infect/contain.cpp
Disease flu(contagiousness,5);
Person joe;
for ( int day{0}; ; ++day ) {
  joe.contact(flu);
  if (joe.is_recovered())
    break;
  joe.one_more_day();
}
```

Note that the person continually comes in contact with the disease. Howver, once sick they don't get re-infected, and once recovered they have built up antibodies and do not get sick from more contact.

Exercise 49.2. Extend the `Person` class with methods that will make the above scenarios work.

The most imporant method is the `one_more_day` method, which models a day progressing in the life of the person.

It is easy to write code that seems to do the right thing, but does not behave correctly in all cases. So it is a good idea to subject your code to some systematic tests.

Make sure your `Person` objects pass these tests:

- After being infected with a 100% transmittable disease, they should register as sick.
- If they are vaccinated or recovered, and they come in contact with such a disease, they stay in their original state.
- If a disease has a transmission chance of 50%, and a number of people come into contact with it, about half of them should get sick. This test maybe a little tricky to write.

Can you use the *Catch2* unit testing framework? See section 69.2.

49.2.2 Interaction

Next we model interactions between people: one person is healthy, another is infected, and when the two come into contact the disease may be transferred.

Adapt the above tests, but now a person comes in contact with an infected person, rather than directly with a disease.

```
// infect/contain.cpp
  Person sick; sick.contact( Disease(1.,5) );
/* ... */
    joe.contact(sick);
```

Exercise 49.3. Bonus: can you get the following disease specification to work?

```
// infect/interaction.cpp
Disease flu;
flu.duration() = 20;
```

```
flu.transfer_probability() = p;
```

Why could you consider this better than the earlier suggested syntax?

49.2.3 Population

Next we need a `Population` class. There is any number of ways we can model a population and how the people in it are connected. For now we will order people as a two-dimensional array. It's up to you how to implement this, but the essential part is that you need to have an indexing operator

```
// infect/contain_lib.hpp
Person& operator[](int i, int j);
```

The `Population` class should at least have the following methods:

- `random_infection` to start out with an infected segment of the population with a given disease;
- `random_vaccination` to start out with a certain fraction of vaccinated individuals.

```
// infect/contain.cpp
Population pop(siz);
pop.random_infection(flu);
pop.vaccinate(vaccination_rate);
```

To run a realistic simulation you also need a `one_more_day` method that takes the population through a day. This is the heart of your code, and we will develop this gradually in the next section.

49.2.3.1 Population tests

Ultimately, you will need functions to test the state of the population:

```
// infect/contain.cpp
Population pop(siz);
pop.random_infection(flu);
pop.vaccinate(vaccination_rate);
```

Most population testing will be done in the following section. For now, make sure you pass the following tests:

- The initial population should be healthy.
- With a vaccination percentage of 100%, everyone should indeed be vaccinated.
- A population 100% infected with a disease should be recovered after a few days; recall that we rule out re-infection.

49.3 Epidemic simulation

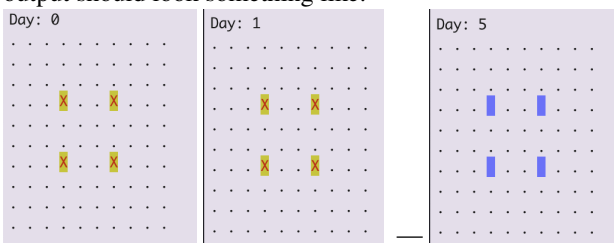
To simulate the spread of a disease through the population, we need an `one_more_day` method that progresses the population through one day:

- Sick people come into contact with a number of other members of the populace;
- and everyone gets one day older, meaning mostly that sick people get one day closer to recovery.

We develop this in a couple of steps.

49.3.1 No contact

At first assume that people have no contact, so the disease ends with the people it starts with. The trace output should look something like:



Remark *Such a display is good for a sanity check on your program behavior. If you include such displays in your writeup, make sure to use a monospace font, and don't use a population size that needs line wrapping. In further testing, you should use large populations, but do not include these displays.*

Exercise 49.4. Program a population with infection.

- Write the `Population` class. The constructor takes the number of people:

  ```
  // infect/contain.cpp
  Population people(siz);
  people.set_transmission_type( Population::transmission_type::none );
  ```

  where the population is a 2D square array of `Person` objects; the input parameter is the length of the side of the square.
- Write a method that infects a number of random people:

  ```
  // infect/contain.cpp
  const int days{5};
  Disease flu(0.f,days);
  people.random_infection( flu );
  ```

- Write a `one_more_day` method:

  ```
  // infect/contain.cpp
  for ( int day=0; day<=days; ++day ) {
    people.display( format( "Day: {}",day ) );
    people.one_more_day();
  }
  ```

  Initially it only has to progress the people in the population; the `Population::one_more_day` method should apply `Person::one_more_day` to all person in the population. There is no transmission of disease yet.

Write a routine that displays the state of the population, using for instance: `?` for susceptible, `+` for infected, `-` for recovered.

49.3.1.1 Tests

Test that for the duration of the disease, the number of infected people stays constant, and that the sum of healthy and infected people stays equal to the population size. Also, the recovery period for the population should be the same as for a single person.

49.3.2 Contagion

This past exercise was too simplistic: the original patient(s) zero was the only one who ever got sick. Now let's incorporate contagion, and investigate the spread of the disease from a single infected person.

We start with a very simple model of infection.

Exercise 49.5. Write a simulation where in each step the direct neighbors of an infected person can now get sick themselves.

Run a number of simulations with population sizes and contagion probabilities. Are there cases where people escape getting sick?

49.3.2.1 Tests

Do some sanity tests:

- If one person is infected with a disease with $p = 1$, the next day there should be 3 people sick.
- If person at the edge of the population is infected, and $p = 1$, the simulation should run for a number of days equal to the square root of the size of the population.
- How is the previous case if $p = 0.5$?

49.3.3 Vaccination

Exercise 49.6. Incorporate vaccination: read another number representing the percentage of people that has been vaccinated. Choose those members of the population randomly.

Describe the effect of vaccinated people on the spread of the disease. Why is this model unrealistic?

49.3.4 Spreading

To make the simulation more realistic, we let every sick person come into contact with a fixed number of random people every day. This gives us more or less the *SIR model* [26].

Set the number of people that a person comes into contact with, per day, to 6 or so. (You can also let this be an upper bound for a random value, but that does not essentially change the simulation.) You have already programmed the probability that a person who comes in contact with an infected person gets sick themselves. Again start the simulation with a single infected person.

Exercise 49.7. Code the random interactions. Now run a number of simulations varying

- The percentage of people vaccinated, and
- the chance the disease is transmitted on contact.

Record how long the disease runs through the population. With a fixed number of contacts and probability of transmission, how is this number of function of the percentage that is vaccinated?

Report this function as a table or graph. Make sure you have enough data points for a meaningful conclusion. Use a realistic population size. You can also do multiple runs and report the average, to even out the effect of the random number generator.

Exercise 49.8. Investigate the matter of 'herd immunity': if enough people are vaccinated, then some people who are not vaccinated will still never get sick. Let's say you want to have the probability of being not vaccinated, yet never getting sick, to be over 95 percent. Investigate the percentage of vaccination that is needed for this as a function of the contagiousness of the disease.

As in the previous exercise, make sure your data set is large enough.

Remark *The screen output you used above is good for sanity checks on small problems. However, for realistic simulations you have to think what is a realistic population size. If your university campus is a population where random people are likely to meet each other, what would be a population size to model that? How about the city where you live?*

Likewise, if you test different vaccination rates, what granularity do you use? With increases of 5 or 10 percent you can print all results to you screen, but you may miss things. Don't be afraid to generate large amount of data and feed them directly to a graphing program.

49.3.4.1 Sanity tests

As your model gets more realistic it becomes harder to test it fully, if only because of the use of random numbers. Still ...

- With zero vaccination, the disease should run for some logarithm of the size of the population.
- With hundred percent vaccination, apart from one sick person, the disease will run for as long as that person is sick.

49.4 Visualization

For debugging purposes it is useful to have visual output. Write a display routine with an optional header:

```
// infect/contain_lib.hpp
void display( std::string caption="" ) const;
```

You can turn the output into an animation by using simple VT100 cursor control:

- Insert a delay after you display one day:

  ```
  #include <thread>
  std::this_thread::sleep_for( std::chrono::milliseconds(200) );
  ```

- The following black magic erases the screen and resets the output to top left of the screen:

  ```
  std::cout << std::format("\u{1b}[2J");
  std::cout << std::format("\u{1b}[0;0H");
  ```

You can also explore coloring the individual persons; see section 27.2 for further links.

49.5 More sophisticated tests

49.5.1 Mutation

The Covid years have shown how important mutations of an original virus can be. Next, you can include mutation in your project. We model this as follows:

- Every so many transmissions, a virus will mutate into a new variant.
- A person who has recovered from one variant is still susceptible to other variants.
- For simplicity assume that each variant leaves a person sick the same number of days, and
- Vaccination is all-or-nothing: one vaccine is enough to protect against all variant;
- On the other hand, having recovered from one variant is not protection against others.

Implementation-wise speaking, we model this as follows. First of all, we need a `Disease` class, so that we can infect a person with an explicit virus;

```
// infect/infect_lib.hpp
void touch( const Person&,long int ps=0 );
void infect( const Disease& );
```

A `Disease` object now carries the information such as the chance of transmission, or how a long a person stays under the weather. Modeling mutation is a little tricky. You could do it as follows:

- There is a global `variants` counter for new virus variants, and a global `transmissions` counter.
- Every time a person infects another, the newly infected person gets a new `Disease` object, with the current variant, and the transmissions counter is updated.
- There is a parameter that determines after how many transmissions the disease mutates. If there is a mutation, the global `variants` counter is updated, and from that point on, every infection is with the new variant. (Note: this is not very realistic. You are free to come up with a better model.)
- A each `Person` object has a vector of variants that they are recovered from; recovery from one variant only makes them immune from that specific variant, not from others.

Exercise 49.9. Add mutation to your model. Experiment with the mutation rate: as the mutation rate increases, the disease should stay in the population longer. Does the relation with vaccination rate change that you observed before?

49.5.1.1 Tests

Implement some sanity tests.

Suppose a person never recovers from a disease (can you think of examples?). The number of mutations should increase roughly exponentially (why isn't the rate actually exponential?); test this.

49.5.2 Diseases without vaccine: Ebola and Covid-19

This section is optional, for bonus points

The project so far applies to diseases for which a vaccine is available, such as MMR for measles, mumps and rubella. The analysis becomes different if no vaccine exists, such as is the case for *Ebola* and, initially, *Covid-19*.

Instead, you need to incorporate 'social distancing' into your code: people do not get in touch with random others anymore, but only those in a very limited social circle. Design a model distance function, and explore various settings.

The difference between Ebola and Covid-19 is how long an infection can go unnoticed: the *incubation period*. With Ebola, infection is almost immediately apparent, so such people are removed from the

general population and treated in a hospital. For Covid-19, a person can be infected, and infect others, for a number of days before they are sequestered from the population.

Add this parameter to your simulation and explore the behavior of the disease as a function of it.

49.6 Ethics

The subject of infectious diseases and vaccination is full of ethical questions. The main one is *The chances of something happening to me are very small, so why shouldn't I bend the rules a little?*. This reasoning is most often applied to vaccination, where people for some reason or other refuse to get vaccinated.

Explore this question and others you may come up with: it is clear that everyone bending the rules will have disastrous consequences, but what if only a few people did this?

49.7 Bonus: parallelism

The main computation in your code is the update of the population. Argue that this is a fully parallel operation. Use OpenMP to make the loop indeed parallel, and test the speedup you get.

Can you also express this as a range algorithm? Again test the speedup you get. On high core counts you may need to figure out the equivalent of `OMP_PROC_BIND=true`. This depends on how your compiler has implemented the parallel range algorithsm.

49.8 Bonus: testing

Read [10]. Can you test the hypothesis of that article?

49.9 Bonus: mathematical analysis

The SIR model can also be modeled through coupled difference or differential equations.

1. The number S_i of susceptible people at time i decreases by a fraction

   $$S_{i+1} = S_i(1 - \lambda_i \, dt)$$

   where λ_i is the product of the number of infected people and a constant that reflects the number of meetings and the infectiousness of the disease. We write:

   $$S_{i+1} = S_i(1 - \lambda I_i \, dt)$$

2. The number of infected people similarly increases by $\lambda S_i I_i$, but it also decreases by people recovering (or dying):

   $$I_{i+1} = I_i(1 + \lambda S_i \, dt - \gamma \, dt).$$

3. Finally, the number of 'removed' people equals that last term:

   $$R_{i+1} = R_i(1 + \gamma I_i).$$

Exercise 49.10. Code this scheme. What is the effect of varying dt?

Exercise 49.11. For the disease to become an epidemic, the number of newly infected has to be larger than the number of recovered. That is,

$$\lambda S_i I_i - \gamma I_i > 0 \Leftrightarrow S_i > \gamma/\lambda.$$

Can you observe this in your simulations?

The parameter γ has a simple interpretation. Suppose that a person stays ill for δ days before recovering. If I_t is relatively stable, that means every day the same number of people get infected as recover, and therefore a $1/\delta$ fraction of people recover each day. Thus, γ is the reciprocal of the duration of the infection in a given person.

49.10 Project writeup and submission

49.10.1 Program files

In the course of this project you have written more than one main program, but some code is shared between the multiple programs. Organize your code with one file for each main program, and a single 'library' file with the class methods.

You can do this two ways:

1. You make a 'library' file, say `infect_lib.cc`, and your main programs each have a line

   ```
   #include "infect_lib.cc"
   ```

   This is not the best solution, but it is acceptable for now.
2. The better solution requires you to use *separate compilation* for building the program, and you need a *header* file. You would now have `infect_lib.cc` which is compiled separately, and `infect_lib.h` which is included both in the library file and the main program:

   ```
   #include "infect_lib.h"
   ```

   See section 19.2.2 for more information.

Submit all source files with a `CMakeLists.txt` file to build it.

49.10.2 Writeup

In the writeup, describe the 'experiments' you have performed and the conclusions you draw from them. The exercises above give you a number of questions to address.

For each main program, include some sample output, but note that this is no substitute for writing out your conclusions in full sentences.

The exercises in section 49.3.4 ask you to explore the program behavior as a function of one or more parameters. Include a table to report on the behavior you found. You can use Matlab or Matplotlib in Python (or even Excell) to plot your data, but that is not required.

Chapter 50

Google PageRank

50.1 Basic ideas

We are going to simulate the Internet. In particular, we are going to simulate the *Google Pagerank* algorithm by which *Google* determines the importance of web pages.

Let's start with some basic classes:

- A `Page` contains some information such as its title and a global numbering in Google's data-center. It also contains a collection of links.
- We represent a link with a pointer to a `Page`. Conceivably we could have a `Link` class, containing further information such as probability of being clicked, or number of times clicked, but for now a pointer will do.
- Ultimately we want to have a class `Web` which contains a number of pages and their links. The web object will ultimately also contain information such as relative importance of the pages.

This application is a natural one for using pointers. When you click on a link on a web page you go from looking at one page in your browser to looking at another. You could implement this by having a pointer to a page, and clicking updates the value of this pointer.

Exercise 50.1. Make a class `Page` which initially just contains the name of the page. Write a method to display the page. Since we will be using pointers quite a bit, let this be the intended code for testing:

```
// google/web2.cpp
auto homepage = make_shared<Page>("My Home Page");
cout << "Homepage has no links yet:" << '\n';
cout << homepage->as_string() << '\n';
```

Next, add links to the page. A link is a pointer to another page, and since there can be any number of them, you will need a `vector` of them. Write a method `click` that follows the link. Intended code:

```
// google/web2.cpp
auto utexas = make_shared<Page>("University Home Page");
homepage->add_link(utexas);
auto searchpage = make_shared<Page>("google");
homepage->add_link(searchpage);
cout << homepage->as_string() << '\n';
```

Exercise 50.2. Add some more links to your homepage. Write a method `random_click` for the `Page` class. Intended code:

```
// google/web2.cpp
for (int iclick=0; iclick<20; ++iclick) {
  auto newpage = homepage->random_click();
  cout << "To: " << newpage->as_string() << '\n';
}
```

How do you handle the case of a page without links?

50.2 Clicking around

Exercise 50.3. Now make a class `Web` which foremost contains a bunch (technically: a `vector`) of pages. Or rather: of pointers to pages. Since we don't want to build a whole internet by hand, let's have a method `create_random_links` which makes a random number of links to random pages. Intended code:

```
// google/web2.cpp
Web internet(netsize);
internet.create_random_links(avglinks);
```

Now we can start our simulation. Write a method `Web::random_walk` that takes a page, and the length of the walk, and simulates the result of randomly clicking that many times on the current page. (Current page. Not the starting page.)

Let's start working towards PageRank. First we see if there are pages that are more popular than others. You can do that by starting a random walk once on each page. Or maybe a couple of times.

Exercise 50.4. Apart from the size of your internet, what other design parameters are there for your tests? Can you give a back-of-the-envelope estimation of their effect?

Exercise 50.5. Your first simulation is to start on each page a number of times, and counts where that lands you. Intended code:

```
// google/web2.cpp
    vector<int> landing_counts(internet.number_of_pages(),0);
    for ( auto page : internet.all_pages() ) {
      for (int iwalk=0; iwalk<5; ++iwalk) {
        auto endpage = internet.random_walk(page,2*avglinks,tracing);
        landing_counts.at(endpage->global_ID())++;
      }
    }
```

Display the results and analyze. You may find that you finish on certain pages too many times. What's happening? Fix that.

50.3 Graph algorithms

There are many algorithms that rely on gradually traversing the web. For instance, any graph can be *connected*. You test that by

- Take an arbitrary vertex v. Make a 'reachable set' $R \leftarrow \{v\}$.

- Now see where you can get from your reachable set:

$$\forall_{v\in V}\forall_{w \text{ neighbour of } v}: R \leftarrow R \cup \{w\}$$

- Repeat the previous step until R does not change anymore.

After this algorithm concludes, is R equal to your set of vertices? If so, your graph is called (fully) connected. If not, your graph has multiple *connected components*.

Exercise 50.6. Code the above algorithm, keeping track of how many steps it takes to reach each vertex w. This is the *Single Source Shortest Path* algorithm (for unweighted graphs).

The *diameter* is defined as the maximal shortest path. Code this.

50.4 Page ranking

The Pagerank algorithm now asks, if you keep clicking randomly, what is the distribution of how likely you are to wind up on a certain page. The way we calculate that is with a probability distribution: we assign a probability to each page so that the sum of all probabilities is one. We start with a random distribution:

Code:

```
// google/web2.cpp
ProbabilityDistribution
  random_state(internet.number_of_pages());
random_state.set_random();
cout << "Initial distribution: " <<
    random_state.as_string() << '\n';
```

Output:

```
[google] pdfsetup:
Initial distribution:
    ↪0:0.00, 1:0.02, 2:0.07,
    ↪3:0.05, 4:0.06, 5:0.08,
    ↪6:0.04, 7:0.04, 8:0.04,
    ↪9:0.01, 10:0.07,
    ↪11:0.05, 12:0.01,
    ↪13:0.04, 14:0.08,
    ↪15:0.06, 16:0.10,
    ↪17:0.06, 18:0.11,
    ↪19:0.01,
```

Exercise 50.7. Implement a class `ProbabilityDistribution`, which stores a vector of floating point numbers. Write methods for:

- accessing a specific element,
- setting the whole distribution to random, and
- normalizing so that the sum of the probabilities is 1.
- a method rendering the distribution as string could be useful too.

Next we need a method that given a probability distribution, gives you the new distribution corresponding to performing a single click. (This is related to *Markov chains*; see HPC book [13], section 10.2.1.)

Exercise 50.8. Write the method

```
ProbabilityDistribution Web::globalclick
    ( ProbabilityDistribution currentstate );
```

Test it by

- start with a distribution that is nonzero in exactly one page;
- print the new distribution corresponding to one click;
- do this for several pages and inspect the result visually.

Then start with a random distribution and run a couple of iterations. How fast does the process converge? Compare the result to the random walk exercise above.

Exercise 50.9. In the random walk exercise you had to deal with the fact that some pages have no outgoing links. In that case you transitioned to a random page. That mechanism is lacking in the `globalclick` method. Figure out a way to incorporate this.

Let's simulate some simple 'search engine optimization' trick.

Exercise 50.10. Add a page that you will artificially made look important: add a number of pagesthat all link to this page, but no one links to them. (Because of the random clicking they will still sometimes be reached.)

Compute the rank of the artificially hyped page. Did you manage to trick Google into ranking this page high? How many links did you have to add?

Sample output:

```
Internet has 5000 pages
Top score: 109:0.0013, 3179:0.0012, 4655:0.0010, 3465:0.0009, 4298:0.0008
With fake pages:
Internet has 5051 pages
Top score: 109:0.0013, 3179:0.0012, 4655:0.0010, 5050:0.0010, 4298:0.0008
Hyped page scores at 4
```

50.5 Graphs and linear algebra

The probability distribution is essentially a vector. You can also represent the web as a matrix W with $w_{ij} = 1$ if page i links to page j. (For details see HPC book [13], section 20.5.) How can you interpret the `globalclick` method in these terms?

Exercise 50.11. Add the matrix representation of the `Web` object and reimplement the `globalclick` method. Test for correctness.

Do a timing comparison.

The iteration you did above to find a stable probability distribution corresponds to the 'power method' in linear algebra. Look up the Perron-Frobenius theory and see what it implies for page ranking.

50.6 Bonus: parallelism

Explore the OpenMP library to parallelize the various algorithms you have developed in this project.

Argue that a Compressed Row Storage (CRS) sparse matrix representation is easy to parallelize. How is that for the more direct implementation of a graph as a linked structure?

Chapter 51

Redistricting

In this project you can explore 'gerrymandering', the strategic drawing of districts to give a minority population a majority of districts[1].

51.1 Basic concepts

We are dealing with the following concepts:

- A state is divided into census districts, which are given. Based on census data (income, ethnicity, median age) one can usually make a good guess as to the overall voting in such a district.
- There is a predetermined number of congressional districts, each of which consists of census districts. A congressional district is not a random collection: the census districts have to be contiguous.
- Every couple of years, to account for changing populations, the district boundaries are redrawn. This is known as redistricting.

There is considerable freedom in how redistricting is done: by shifting the boundaries of the (congressional) districts it is possible to give a population that is in the overall minority a majority of districts. This is known as *gerrymandering* [22].

For background reading, see `https://redistrictingonline.org/`.

To do a small-scale computer simulation of gerrymandering, we make some simplifying assumption.

- First of all, we dispense with census district: we assume that a district consists directly of voters, and that we know their affiliation. In practice one relies on proxy measures (such as income and education level) to predict affiliation.
- Next, we assume a one-dimensional state. This is enough to construct examples that bring out the essence of the problem:

  Consider a state of five voters, and we designate their votes as `AAABB`. Assigning them to three (contiguous) districts can be done as `AAA|B|B`, which has one 'A' district and two 'B' districts.
- We also allow districts to be any positive size, as long as the number of districts is fixed.

1. This project is obviously based on the Northern American political system. Hopefully the explanations here are clear enough. Please contact the author if you know of other countries that have a similar system.

51.2 Basic functions

51.2.1 Voters

We dispense with census districts, expressing everything in terms of voters, for which we assume a known voting behavior. Hence, we need a `Voter` class, which will record the voter ID and party affiliation. We assume two parties, and leave an option for being undecided.

Exercise 51.1. Implement a `Voter` class. You could for instance let ± 1 stand for `A/B`, and 0 for undecided.

```
// gerry/linear.cpp
cout << "Voter 5 is positive:" << '\n';
Voter nr5(5,+1);
cout << nr5.print() << '\n';
```

```
// gerry/linear.cpp
cout << "Voter 6 is negative:" << '\n';
Voter nr6(6,-1);
cout << nr6.print() << '\n';
```

```
// gerry/linear.cpp
cout << "Voter 7 is weird:" << '\n';
Voter nr7(7,3);
cout << nr7.print() << '\n';
```

51.2.2 Populations

Exercise 51.2. Implement a `District` class that models a group of voters.

- You probably want to create a district out of a single voter, or a vector of them. Having a constructor that accepts a string representation would be nice too.
- Write methods `majority` to give the exact majority or minority, and `lean` that evaluates whether the district overall counts as A party or B party.
- Write a `sub` method to creates subsets.

  ```
  District District::sub(int first,int last);
  ```

- For debugging and reporting it may be a good idea to have a method

  ```
  string District::print();
  ```

Code:

```
// gerry/linear.cpp
    cout << "Making district with one B voter"
         << '\n';
    Voter nr5(5,+1);
    District nine( nr5 );
    cout << ".. size: " << nine.size() << '\n
    ';
    cout << ".. lean: " << nine.lean() << '\n
    ';
    /* ... */
    cout << "Making district ABA" << '\n';
    District nine( vector<Voter>
                   { {1,-1},{2,+1},{3,-1} } );
    cout << ".. size: " << nine.size() << '\n
    ';
    cout << ".. lean: " << nine.lean() << '\n
    ';
```

Output:

```
[gerry] district:
Making district with one B voter
.. size: 1
.. lean: 1

Making district ABA
.. size: 3
.. lean: -1
```

Exercise 51.3. Implement a `Population` class that will initially model a whole state.

Code:

```
// gerry/linear.cpp
string pns( "-++--" );
Population some(pns);
cout << "Population from string "
     << pns << '\n';
cout << ".. size: " << some.size() << '\n';
cout << ".. lean: " << some.lean() << '\n';
Population group=some.sub(1,3);
cout << "sub population 1--3" << '\n';
cout << ".. size: " << group.size() << '\n';
cout << ".. lean: " << group.lean() << '\n';
```

Output:

```
[gerry] population:
Population from string -++--
.. size: 5
.. lean: -1
sub population 1--3
.. size: 2
.. lean: 1
```

In addition to an explicit creation, also write a constructor that specifies how many people and what the majority is:

```
// gerry/linear.cpp
Population( int population_size,int majority,bool trace=false )
```

Use a random number generator to achieve precisely the indicated majority.

51.2.3 Districting

The next level of complication is to have a set of districts. Since we will be creating this incrementally, we need some methods for extending it.

Exercise 51.4. Write a class `Districting` that stores a `vector` of `District` objects. Write `size` and `lean` methods:

Code:

```
// gerry/linear.cpp
cout << "Making single voter population B" <<
    '\n';
Population people( vector<Voter>{ Voter(0,+1)
    } );
cout << ".. size: " << people.size() << '\n';
cout << ".. lean: " << people.lean() << '\n';

Districting gerry;
cout << "Start with empty districting:" << '\n
    ';
cout << ".. number of districts: "
     << gerry.size() << '\n';
```

Output:

```
[gerry] gerryempty:
Making single voter population B
.. size: 1
.. lean: 1
Start with empty districting:
.. number of districts: 0
```

Exercise 51.5. Write methods to extend a `Districting`:

```
// gerry/linear.cpp
cout << "Add one B voter:" << '\n';
gerry = gerry.extend_with_new_district( people.at(0) );
cout << ".. number of districts: " << gerry.size() << '\n';
cout << ".. lean: " << gerry.lean() << '\n';
cout << "add A A:" << '\n';
gerry = gerry.extend_last_district( Voter(1,-1) );
gerry = gerry.extend_last_district( Voter(2,-1) );
cout << ".. number of districts: " << gerry.size() << '\n';
cout << ".. lean: " << gerry.lean() << '\n';

cout << "Add two B districts:" << '\n';
gerry = gerry.extend_with_new_district( Voter(3,+1) );
gerry = gerry.extend_with_new_district( Voter(4,+1) );
cout << ".. number of districts: " << gerry.size() << '\n';
cout << ".. lean: " << gerry.lean() << '\n';
```

51.3 Strategy

Now we need a method for districting a population:

```
Districting Population::minority_rules( int ndistricts );
```

Rather than generating all possible partitions of the population, we take an incremental approach (this is related to the solution strategy called *dynamic programming*):

- The basic question is to divide a population optimally over n districts;
- We do this recursively by first solving a division of a subpopulation over $n-1$ districts,
- and extending that with the remaining population as one district.

This means that you need to consider all the ways of having the 'remaining' population into one district, and that means that you will have a loop over all ways of splitting the population, outside of your recursion; see figure 51.1.

- For all $p = 0, \ldots n-1$ considering splitting the state into $0, \ldots, p-1$ and $p, \ldots, n-1$.
- Use the best districting of the first group, and make the last group into a single district.

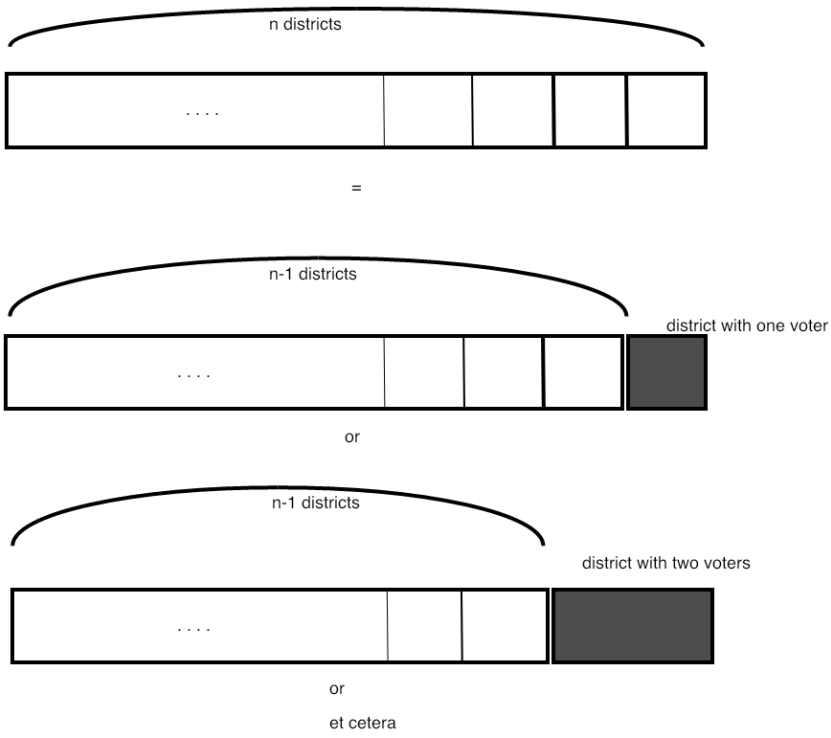


Figure 51.1: Multiple ways of splitting a population

- Keep the districting that gives the strongest minority rule, over all values of p.

You can now realize the above simple example:

```
AAABB => AAA|B|B
```

Exercise 51.6. Implement the above scheme.

Code:

```
// gerry/linear.cpp
Population five("+++--");
cout << "Redistricting population: " << '\n'
     << five.print() << '\n';
cout << ".. majority rule: "
     << five.rule() << '\n';
int ndistricts{3};
auto gerry = five.minority_rules(ndistricts);
cout << gerry.print() << '\n';
cout << ".. minority rule: "
     << gerry.rule() << '\n';
```

Output:

```
[gerry] district5:
Redistricting population:
[0:+,1:+,2:+,3:-,4:-,]
.. majority rule: 1
[3[0:+,1:+,2:+,],[3:-,],[4:-,],]
.. minority rule: -1
```

Note: the range for p given above is not quite correct: for instance, the initial part of the population needs to be big enough to accommodate $n - 1$ voters.

Exercise 51.7. Test multiple population sizes; how much majority can you give party B while still giving party A a majority.

51.4 Efficiency: dynamic programming

If you think about the algorithm you just implemented, you may notice that the districtings of the initial parts get recomputed quite a bit. A strategy for optimizing for this is called *memoization*.

Exercise 51.8. Improve your implementation by storing and reusing results for the initial sub-populations.

In a way, we solved the program backward: we looked at making a district out of the last so-many voters, and then recursively solving a smaller problem for the first however-many voters. But in that process, we decided what is the best way to assign districts to the first 1 voter, first 2, first 3, et cetera. Actually, for more than one voter, say five voters, we found the result on the best attainable minority rule assigning these five voters to one, two, three, four districts.

The process of computing the 'best' districting forward, is known as *dynamic programming*. The fundamental assumption here is that you can use intermediate results and extend them, without having to reconsider the earlier problems.

Consider for instance that you've considered districting ten voters over up to five districts. Now the majority for eleven voters and five districts is the minimum of

- ten voters and five districts, and the new voter is added to the last district; or
- ten voters and four districts, and the new voter becomes a new district.

Exercise 51.9. Code a dynamic programming solution to the redistricting problem.

51.5 Extensions

The project so far has several simplifying assumptions.

- Congressional districts need to be approximately the same size. Can you put a limit on the ratio between sizes? Can the minority still gain a majority?

Exercise 51.10. The biggest assumption is of course that we considered a one-dimensional state. With two dimensions you have more degrees of freedom of shaping the districts. Implement a two-dimensional scheme; use a completely square state, where the census districts form a regular grid. Limit the shape of the congressional districts to be convex.

The *efficiency gap* is a measure of how 'fair' a districting of a state is.

Exercise 51.11. Look up the definition of efficiency gap (and 'wasted votes'), and implement it in your code.

51.6 Ethics

The activity of redistricting was intended to give people a fair representation. In its degenerate form of Gerrymandering this concept of fairness is violated because the explicit goal is to give the minority a majority of votes. Explore ways that this unfairness can be undone.

In your explorations above, the only characteristic of a voter was their preference for party A or B. However, in practice voters can be considered part of communities. The Voting Rights Act is concerned about ‘minority vote dilution’. Can you show examples that a color-blind districting would affect some communities negatively?

Chapter 52

Amazon delivery truck scheduling

This section contains a sequence of exercises that builds up to a simulation of delivery truck scheduling.

52.1 Problem statement

Scheduling the route of a delivery truck is a well-studied problem. For instance, minimizing the total distance that the truck has to travel corresponds to the *Traveling Salesman Problem (TSP)*. However, in the case of *Amazon delivery truck* scheduling the problem has some new aspects:

- A customer is promised a window of days when delivery can take place. Thus, the truck can split the list of places into sublists, with a shorter total distance than going through the list in one sweep.
- Except that *Amazon prime* customers need their deliveries guaranteed the next day.

52.2 Coding up the basics

Before we try finding the best route, let's put the basics in place to have any sort of route at all.

52.2.1 Address list

You probably need a class `Address` that describes the location of a house where a delivery has to be made.

- For simplicity, let give a house (i, j) coordinates.
- We probably need a `distance` function between two addresses. We can either assume that we can travel in a straight line between two houses, or that the city is build on a grid, and you can apply the so-called *Manhattan distance*.
- The address may also require a field recording the last possible delivery date.

Exercise 52.1. Code a class `Address` with the above functionality, and test it.

Code:

```
// amazon/route.cpp
    Address one(1.,1.),
      two(2.,2.);
    cerr << "Distance: "
         << one.distance(two)
         << '\n';
```

Output:

```
[amazon] address:

Address
Distance: 1.41421
.. address
Address 1 should be closest to
    ↪the depot. Check: 1

Route from depot to depot:
    ↪(0,0) (2,0) (1,0) (3,0)
    ↪(0,0)
 has length 8: 8
Greedy scheduling:  (0,0) (1,0)
    ↪(2,0) (3,0) (0,0)
 should have length 6: 6

Square5
Travel in order: 24.1421
Square route:  (0,0) (0,5)
    ↪(5,5) (5,0) (0,0)
 has length 20
.. square5

Original list:  (0,0) (-2,0)
    ↪(-1,0) (1,0) (2,0) (0,0)
 length=8
flip middle two addresses:
    ↪(0,0) (-2,0) (1,0) (-1,0)
    ↪(2,0) (0,0)
 length=12
better:  (0,0) (1,0) (-2,0)
    ↪(-1,0) (2,0) (0,0)
 length=10

Hundred houses
Route in order has length
    ↪25852.6
TSP based on mere listing has
    ↪length: 2751.99 over
    ↪naive 25852.6
Single route has length: 2078.43
 .. new route accepted with
    ↪length 2076.65
Final route has length 2076.65
    ↪over initial 2078.43
TSP route has length 1899.4
    ↪over initial 2078.43

Two routes
Route1:  (0,0) (2,0) (3,2)
    ↪(2,3) (0,2) (0,0)
route2:  (0,0) (3,1) (2,1)
    ↪(1,2) (1,3) (0,0)
total length 19.6251
start with 9.88635,9.73877
Pass 0
.. down to 9.81256,8.57649
Pass 1
Pass 2
Pass 3
Pass 4
TSP Route1:  (0,0) (3,1) (3,2)
    ↪(2,3) (0,2) (0,0)
route2:  (0,0) (2,0) (2,1)
    ↪(1,2) (1,3) (0,0)
total length 18.389
```

Next we need a class `AddressList` that contains a list of addresses.

Exercise 52.2. Implement a class `AddressList`; it probably needs the following methods:

- `add_address` for constructing the list;
- `length` to give the distance one has to travel to visit all addresses in order;
- `index_closest_to` that gives you the address on the list closest to another address, presumably not on the list.

You have options here.

- Do you let your `AddressList` actually contain a `vector<Address>` or do you let it inherit from that container? See section 10.7.
- Another approach is to use a linked list. This has certain advantages; see section 52.6.

Write your code sufficiently abstract that you can accomodate these options.

52.2.2 Add a depot

Next, we model the fact that the route needs to start and end at the depot, which we put arbitrarily at coordinates $(0, 0)$. We could construct an `AddressList` that has the depot as first and last element, but that may run into problems:

- If we reorder the list to minimize the driving distance, the first and last elements may not stay in place.
- We may want elements of a list to be unique: having an address twice means two deliveries at the same address, so the `add_address` method would check that an address is not already in the list.

We can solve this by making a class `Route`, which inherits from `AddressList`, but the methods of which leave the first and last element of the list in place.

52.2.3 Greedy construction of a route

Next we need to construct a route. Rather than solving the full TSP, we start by employing a *greedy search* strategy:

> Given a point, find the next point by some local optimality test, such as shortest distance. Never look back to revisit the route you have constructed so far.

Such a strategy is likely to give an improvement, but most likely will not give the optimal route.

Let's write a method

```
Route::Route greedy_route();
```

that constructs a new address list, containing the same addresses, but arranged to give a shorter length to travel.

Exercise 52.3. Write the `greedy_route` method for the `AddressList` class.

1. Assume that the route starts at the depot, which is located at $(0, 0)$. Then incrementally construct a new list by:
2. Maintain an `Address` variable `we_are_here` of the current location;

3. repeatedly find the address closest to *we_are_here*.

Extend this to a method for the *Route* class by working on the subvector that does not contain the final element.

Test it on this example:

Code:

```
// amazon/route.cpp
Route deliveries;
deliveries.add_address( Address(0,5) );
deliveries.add_address( Address(5,0) );
deliveries.add_address( Address(5,5) );
cerr << "Travel in order: "
     << deliveries.length() << '\n';
assert( deliveries.size()==5 );
auto route = deliveries.greedy_route();
assert( route.size()==5 );
auto len = route.length();
cerr << "Square route:\n "
     << route.as_string()
     << "\n has length " << len << '\n';
```

Output:

```
[amazon] square5:

Travel in order: 24.1421
Square route:  (0,0) (0,5)
    ↪(5,5) (5,0) (0,0)
 has length 20
```

Reorganizing a list can be done in a number of ways.

- First of all, you can try to make the changes in place. This runs into the objection that maybe you want to save the original list; also, while swapping two elements can be done with the **insert** and **erase** methods, more complicated operations are tricky.
- Alternatively, you can incrementally construct a new list. Now the main problem is to keep track of which elements of the original have been processed. You could do this by giving each address a boolean field *done*, but you could also make a copy of the input list, and remove the elements that have been processed. For this, study the **erase** method for **vector** objects.

52.3 Optimizing the route

The above suggestion of each time finding the closest address is known as a *greedy search* strategy. It does not give you the optimal solution of the TSP. Finding the optimal solution of the TSP is hard to program – you could do it recursively – and takes a lot of time as the number of addresses grows. In fact, the TSP is probably the most famous of the class of *NP-hard* problems, which are generally believed to have a running time that grows faster than polynomial in the problem size.

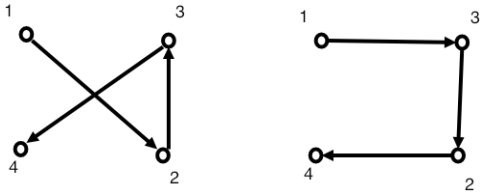


Figure 52.1: Illustration of the 'Opt2' idea of reversing part of a path

However, you can approximate the solution heuristically. One method, the Kernighan-Lin algorithm [25], is based on the *opt2* idea: if you have a path that 'crosses itself', you can make it shorter by reversing part of it. Figure 52.1 shows that the path $1--2--3--4$ can be made shorter by reversing part of it, giving $1--3--2--4$. Since recognizing where a path crosses itself can be hard, or even impossible for graphs that don't have Cartesian coordinates associated, we adopt a scheme where we try all possible reversals:

```
for all nodes m<n on the path [1..N]:
  make a new route from
    [1..m-1] + [m--n].reversed + [n+1..N]
  if the new route is shorter, keep it
```

Exercise 52.4. Code the Opt2 heuristic: write a method to reverse part of the route, and write the loop that tries this with multiple starting and ending points. Try it out on some simple test cases to convince you that your code works as intended.

Let's explore issues of complexity. (For an introduction to complexity calculations, see HPC book [13], section 15.) The TSP is one of a class of *NP complete* problems, which very informally means that there is no better solution than trying out all possibilities.

Exercise 52.5. What is the runtime complexity of the heuristic solution using Opt2? What would the runtime complexity be of finding the best solution by considering all possibilities? Make a very rough estimation of runtimes of the two strategies on some problem sizes: $N = 10, 100, 1000, \ldots$.

Exercise 52.6. (Optional) Reversing sublists is an important operation on your data structure. How many operations does that take? Explore implementing your `AddressList` not as a `vector` but as a `std::list`: a linked list. What is the complexity of reversing a sublist now? Do timing measurements bear this out?

Exercise 52.7. Earlier you had programmed the greedy heuristic. Compare the improvement you get from the Opt2 heuristic, starting both with the given list of addresses, and with a greedy traversal of it.

For realism, how many addresses do you put on your route? How many addresses would a delivery driver do on a typical day?

52.4 Multiple trucks

If we introduce multiple delivery trucks, we get the 'Multiple Traveling Salesman Problem' [5]. With this we can module both the cases of multiple trucks being out on delivery on the same day, or one truck spreading deliveries over multiple days. For now we don't distinguish between the two.

The first question is how to divide up the addresses.

1. We could split the list in two, using some geometric test. This is a good model for the case where multiple trucks are out on the same day. However, if we use this as a model for the same truck being out on multiple days, we are missing the fact that new addresses can be added on the first day, messing up the neatly separated routes.

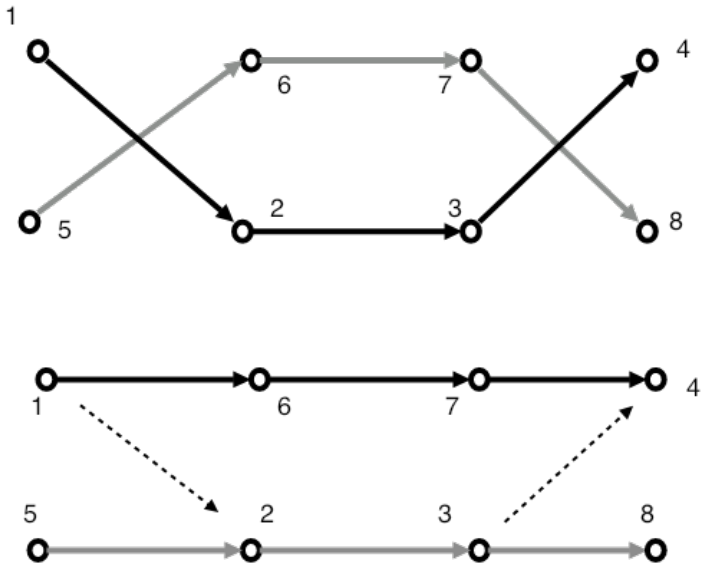


Figure 52.2: Extending the ‘opt2’ idea to multiple paths

2. Thus it may in fact be reasonable to assume that all trucks get an essentially random list of addresses.

Can we extend the Opt2 heuristic to the case of multiple paths? For inspiration take a look at figure 52.2: instead of modifying one path, we could switch bits out bits between one path and another. When you write the code, take into account that the other path may be running backwards! This means that based on split points in the first and second path you know have four resulting modified paths to consider.

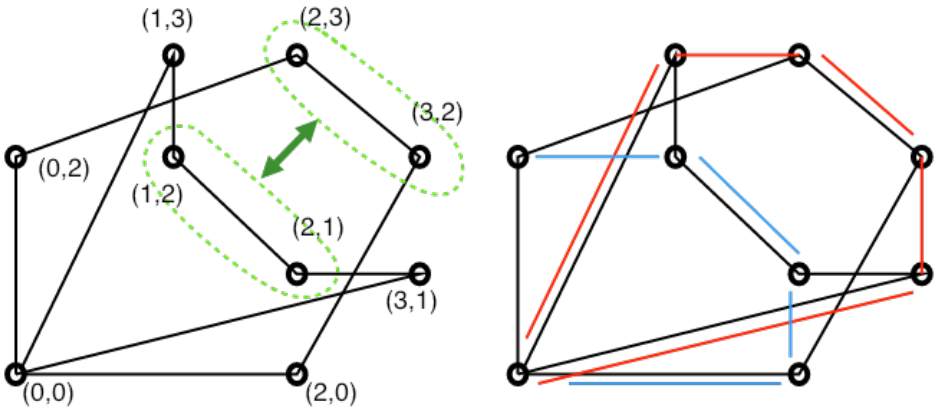


Figure 52.3: Multiple paths test case

Exercise 52.8. Write a function that optimizes two paths simultaneously using the multi-path version of the Opt2 heuristic. For a test case, see figure 52.3.

You have quite a bit of freedom here:

- The start points of the two segments should be chosen independently;
- the lengths can be chosen independently, but need not; and finally
- each segment can be reversed.

More flexibility also means a longer runtime of your program. Does it pay off? Do some tests and report results.

Based on the above description there will be a lot of code duplication. Make sure to introduce functions and methods for various operations.

52.5 Amazon prime

In section 52.4 you made the assumption that it doesn't matter on what day a package is delivered. This changes with *Amazon prime*, where a package has to be delivered guaranteed on the next day.

Exercise 52.9. Explore a scenario where there are two trucks, and each have a number of addresses that can not be exchanged with the other route. How much longer is the total distance? Experiment with the ratio of prime to non-prime addresses.

52.6 Dynamicism

So far we have assumed that the list of addresses to be delivered to is given. This is of course not true: new deliveries will need to be scheduled continuously.

Exercise 52.10. Implement a scenario where every day a random number of new deliveries is added to the list. Explore strategies and design choices.

52.7 Bonus: parallelism

In this project you implemented two algorithms: the greedy strategy, and the Opt2 heuristic. The latter is the easier to parallelize: the Opt2 code explores many possibilities of where to exchange route segments. Implement this with the OpenMP tasking mechanism; MPI/OpenMP book [12], section 24.

The greedy strategy is harder to parallelize. Can you come up with a strategy that is parallel, but may not give quite the same result?

52.8 Ethics

People sometimes criticize Amazon's labor policies, including regarding its drivers. Can you make any observations from your simulations in this respect?

Chapter 53

High performance linear algebra

Linear algebra is fundamental to much of computational science. Applications involving Partial Differential Equations (PDEs) come down to solving large systems of linear equation; solid state physics involves large eigenvalue systems. But even outside of engineering applications linear algebra is important: the major computational part of Deep Learning (DL) networks involves matrix-matrix multiplications.

Linear algebra operations such as the matrix-matrix product are easy to code in a naive way. However, this does not lead to high performance. In these exercises you will explore the basics of a strategy for high performance.

Remark *The algorithm you will develop in this project is an example of* cache-oblivious programming*; see HPC book [13], section 6.9. While this gives considerably higher performance than the naive algorithm, for the highest performance a more sophisticated approach is needed, which involves considerable tuning, as well as a little* assembly coding *[17].*

53.1 Mathematical preliminaries

The matrix-matrix product $C \leftarrow A \cdot B$ is defined as

$$\forall_{ij} : c_{ij} \leftarrow \sum_k a_{ik} b_{kj}.$$

Straightforward code for this would be:

```
for (i=0; i<a.m; i++)
  for (j=0; j<b.n; j++)
    s = 0;
    for (k=0; k<a.n; k++)
      s += a[i,k] * b[k,j];
    c[i,j] = s;
```

However, this is not the only way to code this operation. The loops can be permuted, giving a total of six implementations.

Exercise 53.1. Code one of the permuted algorithms and test its correctness. If the reference algorithm above can be said to be 'inner-product based', how would you describe your variant?

Yet another implementation is based on a block partitioning. Let A, B, C be split on 2×2 block form:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \qquad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \qquad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

Then

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21}, \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22}, \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21}, \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned} \tag{53.1}$$

Convince yourself that this actually computes the same product $C = A \cdot B$. For more on block algorithms, see HPC book [13], section 5.3.6.

Exercise 53.2. Write a matrix class with a multiplication routine:

```
Matrix Matrix::MatMult( Matrix other );
```

First implement a traditional matrix-matrix multiplication, then make it recursive. For the recursive algorithm you need to implement sub-matrix handling: you need to extract submatrices, and write a submatrix back into the surrounding matrix.

Note: this copying is not efficient. Later you will see a better strategy.

53.2 Matrix storage

The simplest way to store an $M \times N$ matrix is as an array of length MN. Inside this array we can decide to store the rows end-to-end, or the columns. While this decision is obviously of practical importance for a library, from a point of performance it makes no difference.

Remark *Historically, linear algebra software such as the BLAS has used columnwise storage, meaning that the location of an element (i, j) is computed as $i+j \cdot M$ (we will use zero-based indexing throughout this project, both for code and mathematical expressions.) The reason for this stems from the origins of the BLAS in the Fortran language, which uses column-major ordering of array elements. On the other hand, C-style arrays (such as `x[5][6]`) in the C/C++ languages have row-major ordering, where element (i, j) is stored in location $j + i \cdot N$.*

Above, you saw the idea of block algorithms, which requires taking submatrices. For efficiency, we don't want to copy elements into a new array, so we want the submatrix to correspond to a subarray.

Now we have a problem: only a submatrix that consists of a sequence of columns is contiguous. The formula $i + j \cdot M$ for location of element (i, j) is no long correct if the matrix is a subblock of a larger matrix.

For this reason, linear algebra software describes a submatrix by three parameters M, N, LDA, where 'LDA' stands for 'leading dimension of A' (see BLAS [21], and Lapack [1]). This is illustrated in figure 53.1.

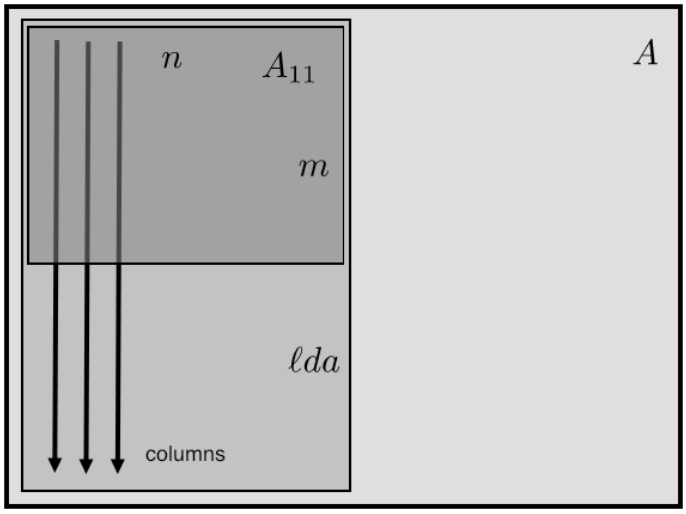


Figure 53.1: Submatrix out of a matrix, with M, N, LDA of the submatrix indicated

Exercise 53.3. In terms of M, N, LDA, what is the location of the (i, j) element?

Implementationwise we also have a problem. If we use `std::vector` for storage, it is not possible to take subarrays, since C++ insists that a vector has its own storage. The solution is to use `span`; section 10.8.3.

Now we have types matrices: top level matrices that store a `vector<double>`, and submatrices that store a `span<double>`. It could be done using `std::variant` (section 24.6.4), but let's not.

Instead, let's adopt the following idiom, where we create a vector at the top level, and then create matrices from its memory.

```
// gemm/lapack_alloc.cpp
// example values for M,LDA,N
M = 2; LDA = M+2; N = 3;
// create a vector to contain the data
vector<double> one_data(LDA*N,1.);
// create a matrix using the vector data
Matrix one(M,LDA,N,one_data.data());
```

Now all types of matrices are created from a `double*`. (If you have not previously programmed in C, you need to get used to the `double*` mechanism. See section 10.10.)

Exercise 53.4. Start implementing the `Matrix` class with a constructor

```
Matrix::Matrix(int m,int lda,int n,double *data)
```

and private data members:

```
// gemm/span_class.hpp
private:
  int m,n,lda;
  gsl::span<double> data;
```

Write a method

```
double& Matrix::at(int i,int j);
```

that you can use as a safe way of accessing elements. Alternatively, overload the index operator.

Exercise 53.5. Do the above exercise, using the C++23 `mdspan` construct.

Let's start with simple operations.

Exercise 53.6. Write a method for adding matrices. Test it on matrices that have the same M, N, but different LDA.

Use of the `at` method is great for debugging, but it is not efficient. Use the preprocessor (chapter 21) to introduce alternatives:

```
#ifdef DEBUG
  c.at(i,j) += a.at(i,k) * b.at(k,j)
#else
  cdata[ /* expression with i,j */ ] += adata[ ... ] * bdata[ ... ]
#endif
```

where you access the data directly with

```
// gemm/span_class.cpp
auto Matrix::get_double_data() {
  double *adata;
  adata = data.data();
  return adata;
};
```

Exercise 53.7. Implement this. Use the `#define` preprocessor directive for the optimized indexing expression. (See section 21.2.2.)

53.2.1 Submatrices

Next we need to support constructing actual submatrices. Since we will mostly aim for decomposition in 2×2 block form, it is enough to write four methods. This can actually be done two different ways:

```
Matrix Left(int j);
Matrix Right(int j);
Matrix Top(int i);
Matrix Bot(int i);
```

```
Matrix TopLeft(int i,int j);
Matrix TopRight(int i,int j);
Matrix BotLeft(int i,int j);
Matrix BotRight(int i,int j);
```

where, for instance, `Left(5)` gives the columns with $j < 5$, or `BotRight(5,6)` give the bottom right block starting at $5, 6$.

Exercise 53.8. Implement these methods and test them.

53.3 Multiplication

You can now write a first multiplication routine, for instance with a prototype

```
void Matrix::MatMult( Matrix& other,Matrix& out );
```

Alternatively, you could write

```
Matrix Matrix::MatMult( Matrix& other );
```

but we want to keep the amount of creation/destruction of objects to a minimum.

53.3.1 One level of blocking

Next, write

```
void Matrix::BlockedMatMult( Matrix& other,Matrix& out );
```

which uses the 2×2 form above.

53.3.2 Recursive blocking

The final step is to make the blocking recursive.

Exercise 53.9. Write a method

```
void RecursiveMatMult( Matrix& other,Matrix& out );
```

which

- Executes the 2×2 block product, using again `RecursiveMatMult` for the blocks.
- When the block is small enough, use the regular `MatMult` product.

53.4 Performance issues

If you experiment a little with the cutoff between the regular and recursive matrix-matrix product, you see that you can get good factor of performance improvement. Why is this?

The matrix-matrix product is a basic operation in scientific computations, and much effort has been put into optimizing it. One interesting fact is that it is just about the most optimizable operation under the sum. The reason for this is, in a nutshell, that it involves $O(N^3)$ operations on $O(N^2)$ data. This means that, in principle each element fetched will be used multiple times, thereby overcoming the *memory bottleneck*.

Exercise 53.10. Make sure that your code has timers; see chapter 24.8. Can you see a performance difference between the naive code and your optimized code? What is the peak speed of your processor, and how close are you getting to it? Is this number dependent on the size of your matrix?

To understand performance issues relating to hardware, you need to do some reading. Section HPC book [13], section 1.3.5 explains the crucial concept of a *cache*.

Exercise 53.11. Argue that the naive matrix-matrix product implementation is unlikely actually to reuse data.

Explain why the recursive strategy does lead to data reuse.

Above, you set a cutoff point for when to switch from the recursive to the regular product.

Exercise 53.12. Argue that continuing to recurse will not have much benefit once the product is contained in the cache. What are the cache sizes of your processor?

Do experiments with various cutoff points. Can you relate this to the cache sizes?

53.5 Bonus: parallelism

The four clauses of equation 53.1 target independent areas in the C matrix, so they could be executed in parallel on any processor that has at least four cores.

Use the 'task' mechanism of the OpenMP library to parallelize the `BlockedMatMult`.

53.6 Bonus: optimal performance

The final question is: how close are you getting to the best possible speed? Unfortunately you are still a way off. You can explore that as follows.

Your computer is likely to have an optimized implementation, accessible through:

```
#include <cblas.h>

cblas_dgemm
  ( CblasColMajor, CblasNoTrans, CblasNoTrans,
    m,other.n,n, alpha,adata,lda,
    bdata,other.lda,
    beta,cdata,out.lda);
```

which computes $C \leftarrow \alpha A \cdot B + \beta C$.

Exercise 53.13. Use another cpp conditional to implement `MatMult` through a call to `cblas_dgemm`. What performance do you now get?

You see that your recursive implementation is faster than the naive one, but not nearly as fast as the CBlas one. This is because

- the CBlas implementation is probably based on an entirely different strategy [17], and
- it probably involves a certain amount of assembly coding.

Chapter 54

The Great Garbage Patch

This section contains a sequence of exercises that builds up to a *cellular automaton* simulation of turtles in the ocean, garbage that is deadly to them, and ships that clean it up. To read more about this: https://theoceancleanup.com/.

Thanks to Ernesto Lima of TACC for the idea and initial code of this exercise.

54.1 Problem and model solution

There is lots of plastic floating around in the ocean, and that is harmful to fish and turtles and cetaceans. Here you get to model the interaction between

- Plastic, randomly located;
- Turtles that swim about; they breed slowly, and they die from ingesting plastic;
- Ships that sweep the ocean to remove plastic debris.

The simulation method we use is that of a *cellular automaton*:

- We have a grid of cells;
- Each cell has a 'state' associated with it, namely, it can contain a ship, a turtle, or plastic, or be empty; and
- On each next time step, the state of a cell is a simple function of the states of that cell and its immediate neighbors.

The purpose of this exercise is to simulate a number of time steps, and explore the iteraction between parameters: with how much gargage will turtles die out, how many ships are enough to protect the turtles.

54.2 Program design

The basic idea is to have an *ocean* object that is populated with turtles, trash, and ships. Your simulation will let the ocean undergo a number of time steps:

```
for (int t=0; t<time_steps; t++)
  ocean.update();
```

Ultimately your purpose is to investigate the development of the turtle population: is it stable, does it die out?

While you can make a 'hackish' solution to this problem, partly you will be judged on your use of modern/clean C++ programming techniques. A number of suggestions are made below.

54.2.1 Grid update

Here is a point to be aware of. Can you see what's wrong with with doing an update entirely in-place:

```
for ( i )
  for ( j )
    cell(i,j) = f( cell(i,j), .... other cells ... );
```

?

54.3 Testing

It can be complicated to test this program for correctness. The best you can do is to try out a number of scenarios. For that it's best to make your program input flexible: use the `cxxopts` package [9], and drive your program from a shell script.

Here is a list of things you can test.

1. Start with only a number of ships; check that after 1000 time steps you still have the same number.
2. Likewise with turtles; if they don't breed and don't die, check that their number stays constant.
3. With only ships and trash, does it all get swept?
4. With only turtles and trash, do they all die off?

It is harder to test that your turtles and ships don't 'teleport' around, but only move to contiguous cells. For that, use visual inspection; see section 27.2.

54.4 Modern programming techniques

54.4.1 Object oriented programming

While there is only have one ocean, you should still make an `ocean` class, rather than having a global array object. All functions are then methods of the single object you create of that class.

54.4.2 Data structure

We lose very little generality by ignoring the depth of the ocean and the shape of coastlines, and model the ocean as a 2D grid.

If you write an indexing function `cell(i,j)` you can make your code largely independent of the actual data structure chosen. Argue why `vector<vector<int>>` is not the best storage.

1. What do you use instead?
2. When you have a working code, can you show by timing that your choice is indeed superior?

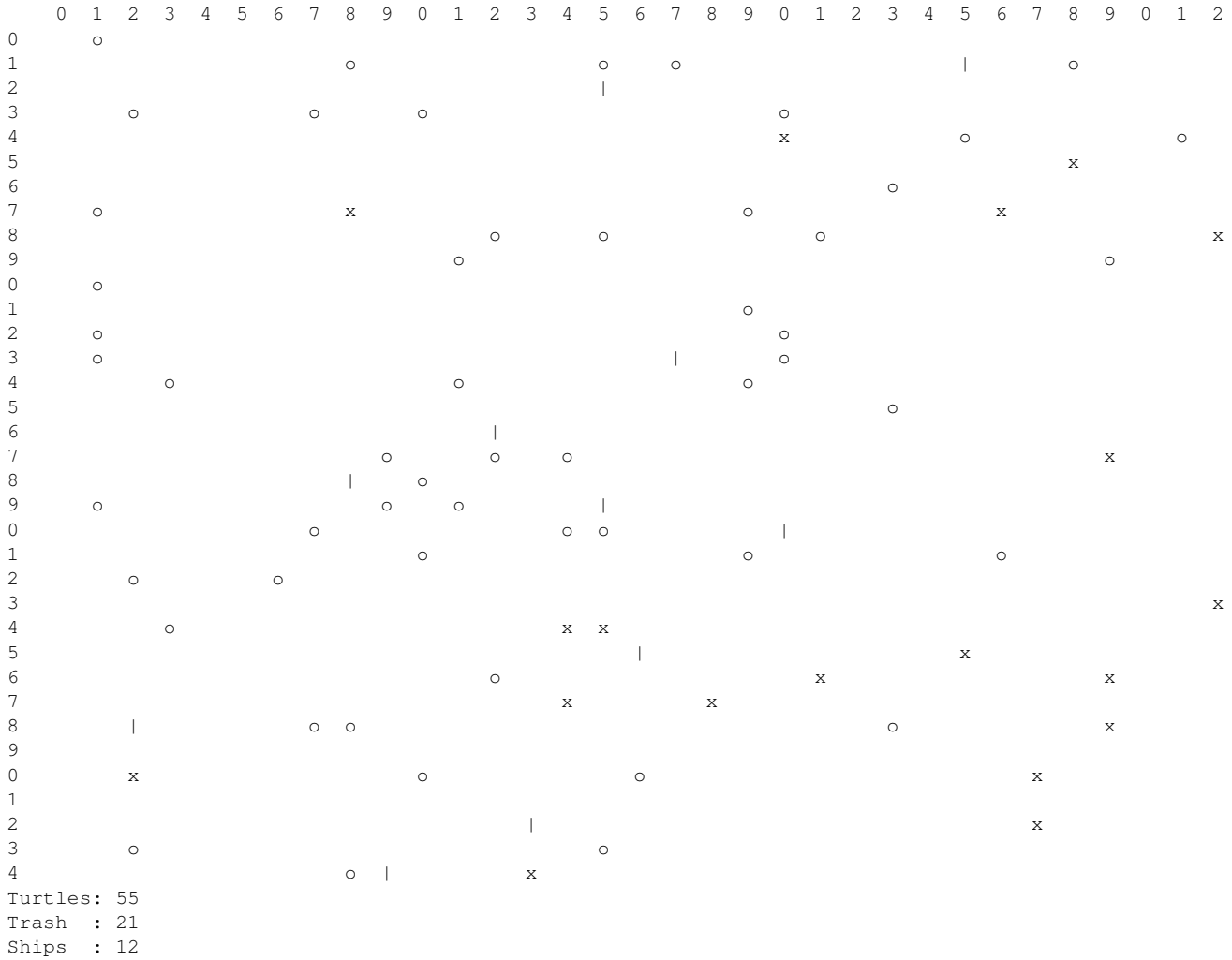


Figure 54.1: Ascii art printout of a time step

54.4.3 Cell types

Having 'magic numbers' in your code (0 =empty, 1 =turtle, et cetera) is not elegant. Make a `enum` or `enum class` (see section 24.10) so that you have names for what's in your cells:

```
// turtles/pacific.cpp
cell(i,j) = occupy::turtle;
```

If you want to print out your ocean, it might be nice if you can directly `cout` your cells:

```
// turtles/pacific.cpp
for (int i=0; i<iSize; ++i) {
  print( "{} ",i%10 );
  for (int j=0; j<jSize; ++j) {
    const auto c = cell(i,j);
    print( "{:>3}",c );
  }
  print("\n");
}
```

54.4.4 Ranging over the ocean

It is easy enough to write a loop as

```
for (int i=0; i<iSize; i++)
  for (int j=0; j<jSize; j++)
    ... cell(i,j) ...
```

However, it may not be a good idea to always sweep over your domain so orderly. Can you implement this:

```
for ( auto [i,j] : permuted_indices() ) {
    ... cell(i,j) ...
```

? See section 24.5 about *structured binding*.

Likewise, if you need to count how many pieces of trash there are around a turtle, can you get this code to work:

```
// turtles/pacific.cpp
int count_around( int ic,int jc,occupy typ ) const {
  int count=0;
  for ( auto [i,j] : neighbors(ic,jc) ) {
    if (cell(i,j) == typ)
      ++count;
  }
  return count;
};
```

You could implement the neighbors function as

```
vector<pair<int,int>> neighbors(int i, int j);
```

but can you figure out how to do it with ranges? What would be the advantage of that?

54.4.5 Random numbers

For the random movement of ships and turtles you need a random number generator. Do not use the old C generator, but the new `random` one; section 24.7.

Try to find a solution so that you use exactly one generator for all places where you need random numbers. Hint: make the generator `static` in your class.

54.5 Explorations

Instead of having the ships move randomly, can you give them a preferential direction to the closest garbage patch? Does this improve the health of the turtle population?

Can you account for the relative size of ships and turtles by having a ship occupy a 2×2 block in your grid?

So far you have let trash stay in place. What if there are ocean currents? Can you make the trash 'sticky' so that trash particles start moving as a patch if they touch?

Turtles eat sardines. (No they don't.) What happens to the sardine population if turtles die out? Can you come up with parameter values that correspond to a stable ecology or a de-stabilized one?

54.5.1 Code efficiency

Investigate whether your implementation of the `enum` in section 54.4.3 has any effect on timing. Parse the fine print of section 24.10.

You may remark that ranging over a largely empty ocean can be pretty inefficient. You could contemplate keeping an 'active list' of where the turtles et cetera are located, and only looping over that. How would you implement that? Do you expect to see a difference in timing? Do you actually?

How is runtime affected by choosing a vector-of-vectors implementation for the ocean; see section 54.4.2.

Chapter 55

Graph algorithms

In this project you will explore some common graph algorithms, and their various possible implementations. The main theme here will be that the common textbook exposition of algorithms is not necessarily the best way to phrase them computationally.

As background knowledge for this project, you are encouraged to read HPC book [13], chapter 10; for an elementary tutorial on graphs, see HPC book [13], chapter 20.

55.1 Traditional algorithms

We first implement the 'textbook' formulations of two *Single Source Shortest Path (SSSP)* algorithms: on unweighted and then on weighted graphs. In the next section we will then consider formulations that are in terms of linear algebra.

In order to develop the implementations, we start with some necessary preliminaries,

55.1.1 Code preliminaries

55.1.1.1 Adjacency graph

We need a class `Dag` for a Directed Acyclic Graph (DAG):

```
// tree/dijkstra1.cpp
class Dag {
private:
  vector<vector<int>> dag;
public:
  // Make Dag of 'n' nodes, no edges for now.
  Dag( int n )
    : dag( vector<vector<int>>(n) ) {};
```

It's probably a good idea to have a function

```
// tree/dijkstra1.cpp
const auto& neighbors( int i ) const { return dag.at(i); };
```

that, given a node, returns a list of the neighbors of that node.

Exercise 55.1. Finish the `Dag` class. In particular, add a method to generate example graphs:

- For testing the 'circular' graph is often useful: connect edges

$$0 \to 1 \to \cdots \to N-1 \to 0.$$

- It may also be a good idea to have a graph with random edges.

Write a method that gives a textual rendering of the graph.

55.1.1.2 Node sets

The classic formulation of SSSP algorithms, such as the *Dijkstra shortest path algorithm* (see HPC book [13], section 10.1.3) uses sets of nodes that are gradually built up or depleted.

You could implement that as a vector:

```
vector< int > set_of_nodes(nnodes);
for ( int inode=0; inode<nnodes; inode++)
  // mark inode as distance unknown:
  set_of_nodes.at(inode) = inf;
```

where you use some convention, such as negative distance, to indicate that a node has been removed from the set.

However, C++ has an actual `set` container with methods for adding an element, finding it, and removing it; see section 24.3.2. This makes for a more direct expression of our algorithms. In our case, we'd need a set of int/int or int/float pairs for the node number and edge weight, depending on the graph algorithm. (It is also possible to use a `map`, using an `int` as lookup key, and `int` or `float` as values.)

For the unweighted graph we only need a set of finished nodes, and we insert node 0 as our starting point:

```
// tree/queuelevel.cpp
using node_info = std::pair<unsigned,unsigned>;
std::set< node_info > distances;
distances.insert( {0,0} );
```

For Dijkstra's algorithm we need both a set of finished nodes, and nodes that we are still working on. We again set the starting node, and we set the distance for all unprocessed nodes to infinity:

```
// tree/queuedijkstra.cpp
const unsigned inf = std::numeric_limits<unsigned>::max();
using node_info = std::pair<unsigned,unsigned>;
std::set< node_info > distances,to_be_done;

to_be_done.insert( {0,0} );
for (unsigned n=1; n<graph_size; ++n)
  to_be_done.insert( {n,inf} );
```

(Why do we need that second set here, while it was not necessary for the unweighted graph case?)

Exercise 55.2. Write a code fragment that tests if a node is in the `distances` set.

- You can of course write a loop for this. In that case know that iterating over a set gives you the key/value pairs. Use *structured bindings*; section 24.5.

- But it's better to use an 'algorithm', in the technical sense of 'algorithms built into the standard library'. In this case, `find`.
- ... except that with `find` you have to search for an exact key/value pair, and here you want to search: 'is this node in the *distances* set with whatever value'. Use the `find_if` algorithm; section 24.3.2.

55.1.2 Level set algorithm

We start with a simple algorithm: the SSSP algorithm in an unweighted graph; see HPC book [13], section 10.1.1 for details. Equivalently, we find level sets in the graph.

For unweighted graphs, the distance algorithm is fairly simple. Inductively:

- Assume that we have a set of nodes reachable in at most n steps,
- then their neighbors (that are not yet in this set) can be reached in $n+1$ steps.

The algorithm outline is

```
// tree/queuelevel.cpp
for (;;) {
  if (distances.size()==graph_size) break;
  /*
   * Loop over all nodes that are already done
   */
  for ( auto [node,level] : distances ) {
    /*
     * set neighbors of the node to have distance 'level + 1'
     */
    const auto& nbors = graph.neighbors(node);
    for ( auto n : nbors ) {
      /*
       * See if 'n' has a known distance,
       * if not, add to 'distances' with level+1
       */
  /* ... */
        {
          cout << "node " << n << " level " << level+1 << '\n';
          distances.insert( {n,level+1} );
        }
```

Exercise 55.3. Finish the program that computes the SSSP algorithm and test it.

This code has an obvious inefficiency: for each level we iterate through all finished nodes, even if all their neighbors may already have been processed.

Exercise 55.4. Maintain a set of 'current level' nodes, and only investigate these to find the next level. Time the two variants on some large graphs.

55.1.3 Dijkstra's algorithm

In Dijkstra's algorithm we maintain both a set of nodes for which the shortest distance has been found, and one for which we are still determining the shortest distance. Note: a tentative shortest distance for a node may be updated several times, since there may be multiple paths to that node. The 'shortest' path in terms of weights may not be the shortest in number of edges traversed!

The main loop now looks something like:

```
// tree/queuedijkstra.cpp
for (;;) {
  if (to_be_done.size()==0) break;
  /*
  * Find the node with least distance
  */
  /* ... */
  cout << "min: " << nclose << " @ " << dclose << '\n';
  /*
  * Move that node to done,
  */
  to_be_done.erase(closest_node);
  distances.insert( *closest_node );
  /*
  * set neighbors of nclose to have that distance + 1
  */
  const auto& nbors = graph.neighbors(nclose);
  for ( auto n : nbors ) {
    // find 'n' in distances
  /* ... */
      {
        /*
    * if 'n' does not have known distance,
    * find where it occurs in 'to_be_done' and update
    */
  /* ... */
        to_be_done.erase( cfind );
        to_be_done.insert( {n,dclose+1} );
  /* ... */
}
```

(Note that we erase a record in the `to_be_done` set, and then re-insert the same key with a new value. We could have done a simple update if we had used a `map` instead of a `set`.)

The various places where you find nodes in the finished / unfinished sets are up to you to implement. You can use simple loops, or use `find_if` to find the elements matching the node numbers.

Exercise 55.5. Fill in the details of the above outline to realize Dijkstra's algorithm.

55.2 Linear algebra formulation

In this part of the project you will explore how much you make graph algorithms look like linear algebra.

55.2.1 Code preliminaries

55.2.1.1 Data structures

You need a matrix and a vector. The vector is easy:

```
// tree/graphmvpdijkstra.cpp
class Vector {
private:
```

```
  vector<vectorvalue> values;
public:
  Vector( int n )
    : values( n,infinite ) {};
```

For the matrix, use initially a dense matrix:

```
// tree/graphmvpdijkstra.cpp
class AdjacencyMatrix {
private:
  vector<vector<matrixvalue>> adjacency;
public:
  AdjacencyMatrix(int n)
    : adjacency( vector<vector<matrixvalue>>
                 ( n,vector<matrixvalue>(n,empty) ) ) {
  };
```

but later we will optimize that.

Remark *In general it's not a good idea to store a matrix as a vector-of-vectors (see section 10.8.1), but in this case we need to be able to return a matrix row, so it is convenient. Optionally you can explore the use of `mdspan`; section 10.8.3.1.*

55.2.1.2 Matrix vector multiply

Let's write a routine

```
  Vector AdjacencyMatrix::leftmultiply( const Vector& left );
```

This is the simplest solution, but not necessarily the most efficient one, as it creates a new vector object for each matrix-vector multiply.

As explained in the theory background (HPC book [13], section 10.2) graph algorithms can be formulated as matrix-vector multiplications with unusual add/multiply operations. Thus, the core of the multiplication routine could look like

```
// tree/graphmvp.cpp
    for ( int row=0; row<n; ++row ) {
      for ( int col=0; col<n; ++col) {
        result[col] = add( result[col], mult( left[row],adjacency[row][col] ) );
      }
    }
```

55.2.2 Unweighted graphs

Exercise 55.6. Implement the `add` / `mult` routines to make the SSSP algorithm on unweighted graphs work.

55.2.3 Dijkstra's algorithm

As an example, consider the following adjacency matrix:

```
.  1 . .  5
. .  1 . .
. . .  1 .
. . . .  1
 1 . . . .
```

The shortest distance $0 \rightarrow 4$ is 4, but in the first step a larger distance of 5 is discovered. Your algorithm should show an output similar to this for the successive updates to the known shortest distances:

```
Input :  0 . . . .
step 0:  0 1 . . 5
step 1:  0 1 2 . 5
step 2:  0 1 2 3 5
step 3:  0 1 2 3 4
```

Exercise 55.7. Implement new versions of the `add` / `mult` routines to make the matrix-vector multiplication correspond to Dijkstra's algorithm for SSSP on weighted graphs.

55.2.4 Sparse matrices

The matrix data structure described above can be made more compact by storing only nonzero elements. Implement this.

55.3 Bonus: parallelism

Explore the OpenMP library to parallelize the various algorithms you have developed in this project.

Argue that a CRS sparse matrix representation is easy to parallelize. Realize this with OpenMP and measure speedup.

How hard is it to parallelize the implementation of a graphs as node sets, as you started this project? In this case the level sets are key to parallel processing. However, on any but very large graphs you may not see much speedup.

55.4 Further explorations

How elegant can you make your code through operator overloading?

Can you code the all-pairs shortest path algorithm?

Can you extend the SSSP algorithm to also generate the actual paths?

55.5 Tests and reporting

You now have two completely different implementations of some graph algorithms. Generate some large matrices and time the algorithms.

Discuss your findings, paying attention to amount of work performed and amount of memory needed.